SHARDA UNIVERSITY

Beyond Boundaries

Tarush Kumar (2023396588)

Section- CS I

Group 1

Working of Code

1. Import PyTorch

python

Copy code

import torch

- Imports the **PyTorch** library.
- Used for tensors, GPU computation, and deep learning training.

---

2. Create the Optimizer

python

Copy code

optimizer = torch.optim.AdamW(model.parameters(), lr=5e-5)

What this does:

- Uses **AdamW optimizer** (Adam + weight decay).
- model.parameters() → all trainable weights of the model.
- lr=5e-5 → learning rate (very common for transformers).

Why AdamW?

- Better regularization.
- Prevents overfitting.
- Standard for NLP & vision models.

---

3. Select Device (CPU or GPU)

python

Copy code

device = "cuda" if torch.cuda.is_available() else "cpu" model.to(device)

What happens:

- Checks if **GPU (CUDA)** is available.
- Moves model to:
    - GPU → faster training
    - CPU → slower fallback

Model and data **must be on the same device**.

---

4. Set Model to Training Mode

python

Copy code

model.train()

Enables:

- Dropout
- Batch normalization
- Gradient updates

Without this, the model won't train properly.

---

5. Training Loop (Epochs)

python

Copy code

for epoch in range(50):

- Runs **50 epochs**
- One epoch = one full pass over the training dataset

---

6. Loop Over Training Batches

python

Copy code

for idx, batch in enumerate(train_dataloader):

- train_dataloader yields **mini-batches**
- Each batch improves memory efficiency

---

7. Extract Inputs from Batch

python

Copy code

input_ids = batch.pop("input_ids").to(device) pixel_values = batch.pop("pixel_values").to(device)

What this means:

- input_ids → tokenized text (NLP input)
- pixel_values → image tensors (vision input)

- .pop() removes these from the batch dictionary
- .to(device) moves data to GPU/CPU

👉 This suggests a **multimodal model** (text + image), like **BLIP, CLIP, or LLaVA**.

---

8. Forward Pass

python

Copy code

```
outputs = model( input_ids=input_ids, pixel_values=pixel_values, labels=input_ids )
```

What happens internally:

- Model processes:
  - Text (input_ids)
  - Image (pixel_values)
- labels=input_ids → model learns to predict the same text
- Used for **self-supervised / captioning / language modeling**

---

9. Compute Loss

python

Copy code

```
loss = outputs.loss
```

- Loss measures **how wrong** the model's prediction is.
- Lower loss = better model.

---

10. Print Loss

python

Copy code

```
print("Loss:", loss.item())
```

- Converts tensor → Python number
- Used for monitoring training progress

---

11. Backpropagation

python

Copy code

loss.backward()

- Computes **gradients**
- Determines how each weight contributed to the loss

This is the **core learning step**.

---

12. Update Model Weights

python

Copy code

optimizer.step()

- Adjusts weights using gradients
- Moves model toward lower loss

---

13. Reset Gradients

python

Copy code

optimizer.zero_grad()

- Clears old gradients
- Prevents gradient accumulation